

ATLAS

Complete Revision & Engineering Guide

A resource book for revising the Go load balancer, understanding its architecture, rebuilding it mentally, and explaining it in interviews.

Topics: reverse proxy • Round Robin • concurrency • health checks • retry/failover • backend control • Prometheus • Grafana • Docker • SSE • real-time dashboard

How to use this book

Use Parts I-X to understand the system, Part XI for testing and debugging, Part XII for design decisions, Part XIII to rebuild Atlas from scratch, and Parts XIV-XVI for interview and rapid revision.

Revision edition — October 2026

Table of Contents

Part	Topic
I	Project Overview
II	Go & HTTP Foundations
III	Load Balancing
IV	Concurrency
V	Health Checking
VI	Retry & Failover
VII	Backend Control
VIII	Observability
IX	Real-Time Dashboard
X	Docker & Deployment
XI	Testing & Debugging
XII	Design Decisions
XIII	Rebuild Atlas From Scratch
XIV	Interview Revision
XV	Rapid Revision Cheat Sheets
XVI	Final Atlas Checklist

Part I — Project Overview

1. What is Atlas?

Atlas is a Go-based HTTP load balancer and reverse proxy. It accepts client requests, selects an eligible backend, forwards traffic, monitors backend health, retries failed requests when possible, exposes operational controls and metrics, and provides a real-time dashboard.

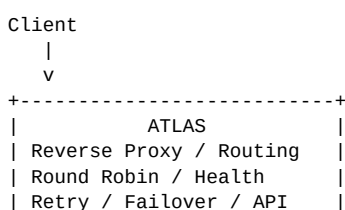
2. What problem does Atlas solve?

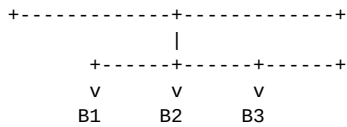
With several backend instances, sending all traffic to one instance creates a bottleneck and a single point of failure. Atlas distributes traffic while removing unhealthy or intentionally disabled backends from routing.

3. Core goals

- Distribute HTTP traffic across multiple backends.
- Avoid unhealthy or administratively disabled backends.
- Recover from backend failures through health checks and retry/failover.
- Expose useful Prometheus metrics and Grafana dashboards.
- Provide a dashboard for visibility and backend control.
- Run reproducibly with Docker Compose.

4. High-level architecture





Atlas ---> Prometheus ---> Grafana
 |
 +--> Event Broker --> SSE --> Dashboard

5. Request lifecycle

- Receive HTTP request.
- Increment request metrics.
- Select backend where Alive && Enabled.
- Reverse-proxy the request.
- Update backend request/latency statistics on success.
- If proxying fails, attempt another eligible backend.
- Return 503 when service cannot be provided.
- Broadcast fresh dashboard status through SSE.

6. Core state model

State	Meaning	Owner
Alive	Current health-check result.	Health checker
Enabled	Operator/admin intent to allow routing.	Control API
Eligible	Alive AND Enabled.	Routing logic

Part II — Go & HTTP Foundations

7. Why Go?

Atlas benefits from Go's standard HTTP library, simple concurrency primitives, efficient network services, static compilation, and low operational overhead.

8. net/http mental model

```
http.Server
|
+-- receives HTTP requests
+-- dispatches them to a Handler

Handler
|
+-- ServeHTTP(ResponseWriter, *Request)
```

Atlas itself can act as an HTTP handler while also exposing dedicated handlers for dashboard, API, metrics, and SSE endpoints.

9. Reverse proxy

Atlas uses net/http/httputil reverse-proxy functionality to forward a request to the selected backend. This lets Atlas focus on selection, health, failure handling, metrics, and control instead of manually rebuilding HTTP forwarding.

10. HTTP client and health checks

Health checks use an HTTP client with a timeout. A timeout prevents a stalled backend from blocking health-check progress indefinitely.

11. Contexts

Request contexts are especially important for SSE. When a browser disconnects, the request context is cancelled, allowing the SSE handler to unsubscribe the client and return.

12. HTTP concepts to revise

- HTTP methods, URLs, headers, bodies, and status codes.
- 2xx/4xx/5xx semantics.
- Timeouts and bounded network operations.
- Reverse proxy versus redirect.
- Long-lived HTTP responses such as SSE.

Part III — Load Balancing

13. Backend model

```
type Backend struct {  
    URL          *url.URL  
    Alive         bool  
    Enabled       bool  
    Requests      int  
    TotalLatency time.Duration  
}
```

Alive and Enabled represent operational state. Requests and TotalLatency support dashboard statistics such as average latency.

14. Round Robin

Round Robin cycles through the backend list, advancing a current position and skipping backends that are not eligible.

Backend 1 -> Backend 2 -> Backend 3 -> Backend 1 -> ...

15. Eligibility

```
eligible = Alive && Enabled
```

A healthy backend may be intentionally disabled. Keeping the two states separate preserves that distinction.

16. Why synchronization?

Requests, health checks, and control operations can access the backend pool concurrently. Shared mutable state therefore needs synchronization.

17. Lock scope

Protect shared state, not network I/O. Atlas should not hold its backend-management lock while waiting for a backend response.

Part IV — Concurrency

18. Mutex versus RWMutex

Primitive	Meaning
sync.Mutex	Exclusive access for reads and writes.
sync.RWMutex	Multiple readers can proceed together; writers are exclusive.

Atlas uses RWMutex because backend state is frequently read and occasionally modified.

19. Critical-section principle

Acquire the lock, perform the smallest required shared-state operation, and release it. Avoid putting slow or unpredictable network calls inside the critical section.

20. Atlas concurrency map

```
HTTP requests  ----Health checker -----> shared backend state --> RWMutex
Control API    -----/
```

21. Race detection

```
go test -race ./...
```

The race detector can reveal unsynchronized concurrent memory access that ordinary functional testing may miss.

22. Common mistakes

- Reading shared fields without the appropriate lock.
- Holding a lock during network I/O.
- Closing a channel while another goroutine may send.
- Trusting stale browser state as authoritative server state.

Part V — Health Checking

23. Purpose

Health checking allows Atlas to detect backends that should temporarily be removed from routing.

24. Active health-check flow

```
Every interval
-> request backend /health
-> status OK?
-> Alive = true/false
-> update healthy-backend metric
-> broadcast status
```

25. Timeout

A bounded health-check timeout prevents one unresponsive backend from delaying the entire health-check cycle.

26. Recovery

A backend marked unhealthy can later become healthy. Health state therefore changes over time rather than being permanent.

27. Health versus enabled

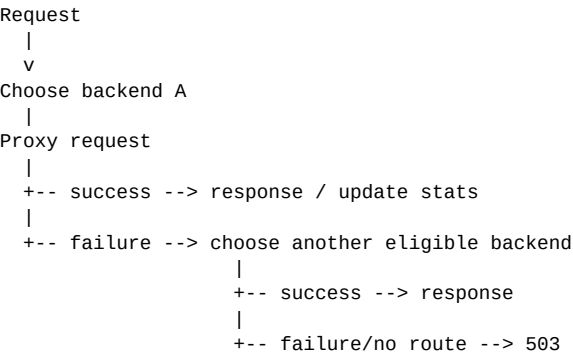
A backend may be healthy but disabled, or enabled but unhealthy. Alive is observed health; Enabled is operator intent.

Part VI — Retry & Failover

28. Why retry?

Health checks are periodic. A backend can fail between two checks. Request-time failure handling covers that gap.

29. Failover flow



30. Bounded retry

Retrying indefinitely can amplify failure and latency. Atlas uses a bounded failover path.

31. Health checks versus retry

Mechanism	Role
Health check	Proactively discovers backend state over time.
Retry/failover	Reacts to request-time failure immediately.

32. 503 behavior

If Atlas cannot find an eligible backend or successfully fail over, it returns HTTP 503 Service Unavailable.

Part VII — Backend Control

33. Control API

```
POST /api/backends/{id}/disable
POST /api/backends/{id}/enable
```

The control path modifies Enabled while the health checker continues to own Alive.

34. Safe state mutation

EnableBackend and DisableBackend validate the ID, lock the load-balancer state, modify Enabled, unlock, and then broadcast the new status.

35. Why broadcast after unlock?

BroadcastStatus obtains a status snapshot through GetStatus, which uses a read lock. Calling it while holding the write lock could deadlock.

36. Multi-tab consistency

The server is authoritative for backend Enabled state. SSE distributes the new status to connected dashboards so multiple tabs converge.

37. Browser-state lesson

A callback can capture an object from the initial /api/status response. That object can become stale after another tab changes server state. The dashboard therefore refreshes current UI state from SSE and uses the current toggle dataset when choosing the next action.

Part VIII — Observability

38. Prometheus metrics

Metric	Type	Purpose
load_balancer_requests_total	Counter	Total requests received.
load_balancer_backend_requests_total	Counter	Requests successfully handled by a backend.
load_balancer_healthy_backends	Gauge	Current healthy backend count.
load_balancer_request_duration_seconds	Histogram	Request-duration distribution.

39. Counter, Gauge, Histogram

Counter: accumulating total. Gauge: current value that can rise or fall. Histogram: distribution of observations into buckets.

40. P95 latency

P95 is the latency value at which roughly 95% of observations are at or below that value. It exposes tail behavior better than a simple average.

41. Grafana

Grafana visualizes Prometheus data. Atlas uses panels for healthy backends, request rate, backend request rate, and P95 request latency. Grafana is observability, not the public Atlas dashboard.

Part IX — Real-Time Dashboard

42. Why SSE?

The dashboard primarily needs server-to-browser updates. Server-Sent Events provide a persistent HTTP stream and are simpler than adding bidirectional WebSocket infrastructure for this use case.

43. EventSource

```
const events = new EventSource("/api/events");

events.onmessage = (event) => {
  const data = JSON.parse(event.data);
  // update dashboard
};
```

44. Event broker

```
Broker
  clients: map[chan []byte]struct{}
  Subscribe()
  Unsubscribe()
  Broadcast()
```

Each connected client receives a buffered channel. Broadcast snapshots the client set under a read lock, releases the lock, then sends to clients.

45. Why snapshot before sending?

Holding the broker mutex during potentially blocking sends would unnecessarily keep other broker operations locked. Snapshotting keeps the critical section small.

46. SSE lifecycle

```
subscribe
-> Content-Type: text/event-stream
-> wait for data or context cancellation
-> write "data: ...\n\n"
-> Flush()
-> on disconnect: unsubscribe and return
```

47. Status broadcasting

Atlas reuses GetStatus as the SSE payload source. Status can be broadcast after health checks, successful request-stat updates, and backend enable/disable changes.

48. SSE vs alternatives

Approach	Atlas fit
Polling	Simple but repetitive and delayed.
SSE	One-way server-to-browser stream; strong fit.
WebSockets	Bidirectional; useful for richer interaction but unnecessary here.

Part X — Docker & Deployment

49. Why Docker?

Docker gives Atlas and its supporting services reproducible environments. Compose starts the load balancer, backend services, Prometheus, and Grafana together.

50. Multi-stage builds

A multi-stage build separates compilation from runtime and allows a smaller final image.

51. Compose architecture

```
docker compose
| - atlas
| - backend 1
| - backend 2
| - backend 3
| - prometheus
` - grafana
```

52. Container networking

Compose services can communicate using service names and container ports inside the Compose network.

53. Public architecture

```
Internet
|
HTTPS / Nginx
|
Atlas :8080
| - dashboard
| - API
` - SSE
```

Prometheus/Grafana -> observability

54. Security reminder

Backend control endpoints change routing state. A real production deployment should protect administrative controls with authentication and authorization. A portfolio demonstration can deliberately keep the scope simpler, but the trade-off should be documented.

Part XI — Testing & Debugging

55. Functional test matrix

Scenario	Expected result
All healthy	Requests rotate across eligible backends.
One unhealthy	Traffic avoids it.
Recovery	Backend can re-enter routing.
Disabled	Traffic avoids it even if healthy.
Re-enabled	Eligible again if healthy.
Selected backend fails	Atlas attempts failover.
No eligible backend	HTTP 503.
SSE connected	Dashboard updates without refresh.
Two dashboard tabs	Backend state converges.

56. Useful verification

```
go test ./...
go test -race ./...
docker compose up --build
```

57. Debugging method

- Find the exact boundary where observed behavior diverges.
- Trace state from input -> server mutation -> broadcast -> browser update.
- Check logs and HTTP status before redesigning.
- For concurrency, inspect ownership and lock scope.
- For SSE, trace state change -> BroadcastStatus -> Broker -> channel -> Flush -> EventSource.
- For Docker, inspect containers, logs, ports, networks, and service names.

58. Lessons

- Working server behavior does not guarantee synchronized browser state.
- State ownership must be explicit.
- Never hold shared-state locks across slow network calls.
- Health checks and request-time failover cover different failure windows.
- Observability is part of operating a system.

Part XII — Design Decisions

59. Why Round Robin?

It is deterministic, simple, and makes routing behavior easy to observe and reason about.

60. Why RWMutex?

Atlas reads shared backend state frequently and modifies it less frequently, so RWMutex expresses the access pattern.

61. Why active health checks?

They proactively discover backend failures. Retry handles failures that occur between checks.

62. Why Alive and Enabled separately?

Observed health and operator intent are different facts and should remain distinguishable.

63. Why SSE?

The dashboard mostly consumes server-originated updates, so one-way streaming is enough.

64. Why an event broker?

Multiple dashboards need the same status updates; the broker centralizes fan-out.

65. Why Prometheus + Grafana?

They provide standard metric collection and visualization.

66. Why Docker Compose?

It makes the multi-service demonstration reproducible.

Decision template

For every major design choice, remember: Problem → Options → Choice → Reason → Trade-offs.

Part XIII — Rebuild Atlas From Scratch

Do not memorize source files. Memorize the dependency chain.

Step	Layer	Mental task
1	HTTP server	Accept requests.
2	Backend model	Represent URL and state.
3	Reverse proxy	Forward to selected backend.
4	Round Robin	Select next eligible backend.
5	Synchronization	Protect shared state.
6	Health checker	Update Alive.
7	Failover	Retry another backend.
8	Controls	Update Enabled.
9	Metrics	Measure requests, health, latency.
10	Prometheus/Grafana	Collect and visualize.
11	Docker Compose	Package the system.
12	Event broker	Fan out status.
13	SSE	Stream to browsers.
14	Dashboard	Render live state.

Mental request trace

```
Client
-> ServeHTTP
-> request metric
-> NextBackend
-> Alive && Enabled
-> ReverseProxy
  | success -> backend stats -> BroadcastStatus
  | failure -> retry another backend
    | success -> stats -> BroadcastStatus
    ` failure -> 503
```

Rebuild exercise

Close the repository. Explain every layer aloud. For each layer answer: What problem does it solve? What state does it own? What can fail? What synchronization is required? What does the next layer depend on?

Part XIV — Interview Revision

67. 60-second explanation

Atlas is a Go-based HTTP load balancer and reverse proxy I built to understand production-style backend systems. It uses Round Robin to distribute traffic across multiple backends, protects shared state with RWMutex, actively health-checks backends, and performs retry/failover when a request encounters a backend failure. I added Prometheus metrics and Grafana for observability, Docker Compose for reproducible deployment, and an SSE-based dashboard for real-time health, traffic, latency, and backend controls.

68. Questions you should be ready for

- 1. Why Go?
- 2. How does `httputil.ReverseProxy` work conceptually?
- 3. How does Round Robin skip unavailable backends?
- 4. Why is synchronization required?
- 5. Why RWMutex instead of Mutex?
- 6. Where could a data race occur?
- 7. Why avoid holding locks during network I/O?
- 8. Why separate Alive and Enabled?
- 9. How does health-check recovery work?
- 10. Why retry if health checks already exist?
- 11. What happens when every backend is unavailable?
- 12. Why use a health-check timeout?
- 13. Counter vs Gauge vs Histogram?
- 14. What does P95 tell you?
- 15. Why SSE instead of WebSockets?
- 16. How does the broker manage clients?
- 17. Why broadcast after unlocking?
- 18. What happens when an SSE client disconnects?
- 19. How would you secure control endpoints?
- 20. How would you add weighted Round Robin?
- 21. How would you scale Atlas horizontally?

69. Diagram questions

- Trace one successful request.
- Trace a first-backend failure and failover.
- Trace unhealthy -> recovered.
- Trace disable from one dashboard tab to another.
- Trace Atlas status through the SSE broker to the browser.
- Trace metrics from Atlas to Prometheus to Grafana.

Part XV — Rapid Revision Cheat Sheets

70. Architecture

```
Client
|
v
Atlas
|- Routing: Round Robin
|- Health: Alive
|- Control: Enabled
|- Failover
`- Metrics
|
+--> Prometheus --> Grafana
|
`--> Broker --> SSE --> Dashboard
```

71. Concurrency

```
RLock -> read shared state
Lock  -> mutate shared state
Unlock -> leave critical section
```

```
Rule:
lock -> tiny state operation -> unlock
```

```
Avoid:
lock -> network request -> wait -> unlock
```

72. Failure handling

```
Health check -> Alive
Request failure -> retry/failover
Routing rule -> Alive && Enabled
No eligible backend -> 503
```

73. SSE

```
State change
-> GetStatus
-> JSON
-> Broadcast
-> client channel
-> SSE handler
-> Flush
-> EventSource.onmessage
```

74. Observability

Term	Remember
Counter	Accumulating total.
Gauge	Current value that can move.
Histogram	Distribution of observations.
P95	95% of observations are at or below the percentile.
Prometheus	Metric collection/time series.
Grafana	Visualization.

Part XVI — Final Atlas Checklist

✓	Item
☐	Explain architecture without opening the repository.
☐	Explain the complete request lifecycle.
☐	Explain Round Robin and Alive && Enabled.
☐	Explain why RWMutex is used.
☐	Explain why locks are not held during network I/O.
☐	Explain health-check timeout and recovery.
☐	Explain retry/failover and 503 behavior.
☐	Explain every Prometheus metric.
☐	Explain P95 latency.
☐	Explain Grafana's role.
☐	Explain the SSE broker and client lifecycle.
☐	Explain SSE versus WebSockets.
☐	Explain Docker Compose networking.
☐	Run go test ./....
☐	Run go test -race ./....
☐	Reproduce backend failure and recovery.
☐	Test enable/disable from multiple tabs.
☐	Deploy the Atlas dashboard.
☐	Update README to match final architecture.
☐	Write the resume entry.
☐	Prepare the LinkedIn project post.

Final principle

Atlas is complete when you can explain not only what the code does, but why each component exists, how state moves through the system, what can fail, how concurrency is controlled, and how the system could evolve. The repository is the implementation; this guide is the mental model.